

Complete Developer Walkthrough

Real-Time UPI Transaction Fraud Detection System

A step-by-step guide covering environment setup, data generation, feature engineering, model training, API development, and dashboard deployment — written for VS Code beginners.

Tool	Level	Duration	Chapters
VS Code + Python 3.11	Beginner-friendly	3–4 Days	7 Chapters

Table of Contents

Chapter 1	Tools & Environment Setup	Installing VS Code, Python, extensions, virtual environment
Chapter 2	Project Structure	Folder layout, file naming, VS Code workspace config
Chapter 3	Data Generation	Faker synthetic UPI dataset + loading PaySim & IEEE-CIS
Chapter 4	Feature Engineering	Tabular features, velocity windows, NetworkX graph features
Chapter 5	Model Training & Explainability	XGBoost, Isolation Forest, SHAP plots, MLflow tracking
Chapter 6	FastAPI Scoring Engine	REST API, /predict endpoint, SHAP response, SQLite logging
Chapter 7	Streamlit Dashboard & Live Stream	Ops dashboard, PyVis graph, simulate_stream.py

Chapter 1

Tools & Environment Setup

Installing VS Code, Python 3.11, extensions, and virtual environment

Why VS Code for this project?

VS Code is the industry-standard editor for Python ML projects. It handles both Jupyter notebooks (for data exploration and model training) and regular .py files (for FastAPI and Streamlit) in a single window. The built-in terminal means you never need to leave VS Code to run commands.

Step-by-Step Installation

1

Install Python 3.11

Go to python.org/downloads and download Python 3.11 (not 3.12+, some ML libraries lag behind). During install on Windows: tick 'Add Python to PATH' — this is critical. On Mac: use the .pkg installer.

2

Install VS Code

Go to code.visualstudio.com and download for your OS (Windows/Mac/Linux). Run the installer with default settings. VS Code is free and made by Microsoft.

3

Install VS Code Extensions

Open VS Code. Click the Extensions icon (left sidebar, looks like 4 squares). Search and install these two: 'Python' by Microsoft, and 'Jupyter' by Microsoft. Both are free and official. Restart VS Code after installing.

4

Create Project Folder

Create a folder on your desktop or Documents called 'upi_fraud_detector'. In VS Code: File → Open Folder → select that folder. This is now your workspace — everything goes inside here.

Setting Up the Virtual Environment

A virtual environment is an isolated Python installation just for this project. It keeps your project's packages separate from your system Python — meaning if you install XGBoost for this project, it won't affect any other Python project on your machine. This is standard practice for every Python ML project.

Why virtual environments matter

Without a virtual environment, all your Python projects share the same packages. Version conflicts between projects will cause errors that are very hard to debug. Always use one — takes 30 seconds to set up.

Open the VS Code terminal: View → Terminal (or Ctrl+` on Windows, Cmd+` on Mac). You'll see a command prompt at the bottom of VS Code. Type these commands one by one:

```
# Navigate into your project folder (should already be there)
cd upi_fraud_detector
```

```
# Create the virtual environment (folder named 'venv')
python -m venv venv

# Activate it – Windows:
venv\Scripts\activate

# Activate it – Mac / Linux:
source venv/bin/activate

# You'll see (venv) appear at the start of your terminal line
# That means it's active. You should see this:
# (venv) C:\Users\you\upi_fraud_detector>
```

Selecting the Python Interpreter in VS Code

VS Code needs to know which Python to use. Press Ctrl+Shift+P (Cmd+Shift+P on Mac) to open the Command Palette. Type 'Python: Select Interpreter' and press Enter. Select the one that shows './venv/bin/python' or './venv\Scripts\python.exe' — that is your virtual environment's Python.

Check it worked

At the bottom-left of VS Code you'll now see the Python version with '(venv)' next to it. If you see this, your interpreter is correctly set.

Install All Required Packages

With your virtual environment active in the terminal, run this single command to install everything the project needs:

```
pip install pandas numpy faker networkx xgboost scikit-learn shap
pip install fastapi uvicorn streamlit pyvis mlflow
pip install matplotlib seaborn plotly ipykernel
```

This takes 3-5 minutes

Pip will download and install all packages. You'll see progress bars for each. If you see any red 'ERROR' lines, copy the error and ask for help — don't continue until all installs succeed.

Verify Everything Installed Correctly

In the terminal, run:

```
python -c "import pandas, numpy, xgboost, shap, fastapi, streamlit"
# If no output appears – everything is installed correctly
# If you see 'ModuleNotFoundError' – that package didn't install, re-run pip install
```

Chapter 2

Project Structure

How to organise files and folders inside VS Code

A clean folder structure makes your project look professional on GitHub and makes it easier to navigate as the codebase grows. Create the following structure inside your `upi_fraud_detector` folder.

```
upi_fraud_detector/  
├──  
│   ├── data/ # All datasets go here  
│   │   ├── raw/ # Downloaded PaySim and IEEE-CIS CSVs  
│   │   └── processed/ # Cleaned, merged feature CSVs  
│   ├──  
│   │   ├── notebooks/ # Jupyter notebooks for exploration  
│   │   │   ├── 01_data_generation.ipynb  
│   │   │   ├── 02_feature_engineering.ipynb  
│   │   │   └── 03_model_training.ipynb  
│   │   ├──  
│   │   │   ├── src/ # Core Python modules  
│   │   │   │   ├── generate_data.py # Faker UPI dataset generator  
│   │   │   │   ├── features.py # Feature engineering functions  
│   │   │   │   └── model.py # Model training & evaluation  
│   │   │   ├──  
│   │   │   │   ├── api/  
│   │   │   │   │   ├── main.py # FastAPI application  
│   │   │   │   ├──  
│   │   │   │   │   ├── dashboard/  
│   │   │   │   │   │   ├── app.py # Streamlit dashboard  
│   │   │   │   │   ├──  
│   │   │   │   │   │   ├── simulate_stream.py # Live transaction simulator  
│   │   │   │   │   │   ├── models/ # Saved model files (.pkl)  
│   │   │   │   │   │   ├── requirements.txt # Package list  
│   │   │   │   │   │   └── README.md # Project documentation
```

Create This Structure Now

In VS Code terminal, run these commands to create all folders at once:

```
mkdir data data/raw data/processed notebooks src api dashboard models
```

```
# Windows users – use this instead:

mkdir data && mkdir data\raw && mkdir data\processed

mkdir notebooks && mkdir src && mkdir api && mkdir dashboard && mkdir models
```

Move Your Downloaded Datasets

Take the PaySim CSV (PS_20174392719_1491204439457_log.csv) and the IEEE-CIS files (train_transaction.csv, train_identity.csv) you downloaded from Kaggle and move them into the data/raw/ folder. That's the only manual step.

Creating files in VS Code

To create a new file: right-click any folder in the VS Code sidebar → 'New File'. To create a Jupyter notebook: right-click notebooks/ → New File → name it '01_data_generation.ipynb'. VS Code will automatically open it as a notebook.

Create requirements.txt

Create a file named requirements.txt in the root folder and paste this inside:

```
pandas==2.1.0
numpy==1.26.0
faker==19.6.2
networkx==3.1
xgboost==2.0.0
scikit-learn==1.3.0
shap==0.43.0
fastapi==0.103.1
uvicorn==0.23.2
streamlit==1.27.0
pyvis==0.3.2
mlflow==2.7.1
matplotlib==3.8.0
seaborn==0.13.0
plotly==5.17.0
```

Chapter 3

Data Generation

Generating the synthetic UPI dataset and loading PaySim + IEEE-CIS

This chapter covers two things: (1) generating your own synthetic UPI dataset using Faker, and (2) loading and previewing the PaySim dataset you downloaded from Kaggle. Create the file `notebooks/01_data_generation.ipynb` and follow along.

Part A — Generate Synthetic UPI Dataset

Open `01_data_generation.ipynb`. Click '+ Code' to add a new cell. Copy each block below into separate cells and run them with Shift+Enter.

Cell 1 — Imports

```
import pandas as pd
import numpy as np
from faker import Faker
import random
from datetime import datetime, timedelta
import os

fake = Faker('en_IN') # Indian locale for realistic names
random.seed(42) # So results are reproducible
np.random.seed(42)
print('Libraries loaded successfully')
```

Cell 2 — Generate 5,000 UPI Accounts

Each account gets a realistic Indian VPA (Virtual Payment Address) and a flag for whether it's a mule account (used for money laundering).

```
UPI_HANDLES = ['@okicici', '@okhdfcbank', '@oksbi', '@okaxis',
               '@paytm', '@ybl', '@upi', '@ibl', '@kotak']

def make_vpa(name):
    # Creates realistic UPI IDs like 'raj.kumar221@okicici'
    clean = name.lower().replace(' ', '.').replace('"', '')
    return clean + str(random.randint(1, 999)) + random.choice(UPI_HANDLES)

accounts = []
n_accounts = 5000
n_mules = int(n_accounts * 0.10) # 10% are mule accounts
```

```
for i in range(n_accounts):
    name = fake.name()
    accounts.append({
        'account_id': f'ACC{i:05d}',
        'vpa': make_vpa(name),
        'name': name,
        'bank': random.choice(['SBI', 'HDFC', 'ICICI', 'Axis', 'Kotak']),
        'account_age_days': random.randint(30, 1825), # 1 month to 5 years
        'is_mule': 1 if i < n_mules else 0,
    })

df_accounts = pd.DataFrame(accounts)
print(f'Accounts created: {len(df_accounts)}')
print(f'Mule accounts: {df_accounts.is_mule.sum()}')
df_accounts.head(3)
```

Cell 3 — Generate 50,000 Transactions

Transactions follow realistic UPI patterns: peak hours, log-normal amounts, and different transaction type distributions.

```
start_date = datetime(2024, 1, 1)
all_txns = []

# Hour weights – UPI peaks at morning and evening
hour_weights = [0.2, 0.1, 0.1, 0.1, 0.2, 0.5, 1.0, 1.5, 2.0, 2.5,
3.0, 2.8, 2.5, 2.2, 2.0, 2.3, 2.5, 2.8, 3.2, 3.5,
3.0, 2.5, 1.5, 0.8]

for i in range(50000):
    sender = df_accounts.sample(1).iloc[0]
    receiver = df_accounts.sample(1).iloc[0]
    while receiver['account_id'] == sender['account_id']:
        receiver = df_accounts.sample(1).iloc[0]

    # Random timestamp over 90 days with realistic hour distribution
    day_offset = random.randint(0, 89)
    hour = random.choices(range(24), weights=hour_weights)[0]
    minute = random.randint(0, 59)
    timestamp = start_date + timedelta(days=day_offset,
        hours=hour, minutes=minute)

    # Log-normal amount distribution (~Rs.850 mean)
    amount = round(np.random.lognormal(mean=6.5, sigma=1.2), 2)
```

```
amount = min(amount, 100000) # UPI limit

txn_type = random.choices(
    ['P2P', 'MERCHANT', 'BILL'],
    weights=[60, 30, 10])[0]

all_txns.append({
    'txn_id': f'TXN{i:07d}',
    'sender_id': sender['account_id'],
    'sender_vpa': sender['vpa'],
    'receiver_id': receiver['account_id'],
    'receiver_vpa': receiver['vpa'],
    'amount': amount,
    'timestamp': timestamp,
    'txn_type': txn_type,
    'is_fraud': 0, # will be updated in next cell
})

df_txns = pd.DataFrame(all_txns).sort_values('timestamp').reset_index(drop=True)

print(f'Transactions generated: {len(df_txns)}')

df_txns.head(3)
```

Cell 4 — Inject Fraud Patterns

Four realistic UPI fraud patterns are injected to create labelled fraud cases.

```
# Pattern A: Mule account receives many small credits then
# sends one large outward transfer
mule_ids = df_accounts[df_accounts.is_mule==1]['account_id'].tolist()
mule_txn_idx = df_txns[df_txns['receiver_id'].isin(mule_ids)].index
df_txns.loc[mule_txn_idx[:2000], 'is_fraud'] = 1

# Pattern B: Velocity spike — same sender, 20+ txns within 1 hour
df_txns['timestamp'] = pd.to_datetime(df_txns['timestamp'])
for acc in df_accounts.sample(30)['account_id']:
    mask = df_txns['sender_id'] == acc
    spike_idx = df_txns[mask].head(25).index
    df_txns.loc[spike_idx, 'timestamp'] = df_txns.loc[spike_idx[0], 'timestamp']
    df_txns.loc[spike_idx, 'is_fraud'] = 1

# Pattern C: Round-number amounts just under Rs.1 lakh limit
round_mask = (df_txns['amount'] >= 99000) & (df_txns['amount'] <= 99999)
df_txns.loc[round_mask, 'is_fraud'] = 1
```



```
fraud_rate = df_txns['is_fraud'].mean() * 100
print(f'Fraud rate: {fraud_rate:.2f}%')
print(f'Fraud txns: {df_txns.is_fraud.sum()}')

# Save to file
os.makedirs('../data/raw', exist_ok=True)
df_txns.to_csv('../data/raw/upi_synthetic.csv', index=False)
df_accounts.to_csv('../data/raw/upi_accounts.csv', index=False)
print('Saved to data/raw/')
```

Part B — Load PaySim Dataset

Now load the PaySim CSV you downloaded from Kaggle and preview it.

```
# Load PaySim — may take 10-20 seconds (6.3M rows)
df_paysim = pd.read_csv('../data/raw/PS_20174392719_1491204439457_log.csv')

print('PaySim shape:', df_paysim.shape)
print('Fraud rate:', df_paysim['isFraud'].mean()*100, '%')
print('\nColumns:', df_paysim.columns.tolist())
df_paysim.head(3)
```

PaySim columns you'll use

step (time), type (CASH-IN/CASH-OUT/TRANSFER etc), amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest, isFraud. The isFraud column (0 or 1) provides your ground-truth labels.

Chapter 4

Feature Engineering

Tabular features, velocity windows, and NetworkX graph features

Feature engineering is the most important step in this project. Good features beat complex models every time. You'll build three types: tabular features from transaction attributes, velocity features from time windows, and graph features from the transaction network. Create notebooks/02_feature_engineering.ipynb.

Cell 1 — Load Data and Imports

```
import pandas as pd
import numpy as np
import networkx as nx
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('../data/raw/upi_synthetic.csv')
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.sort_values('timestamp').reset_index(drop=True)
print('Data loaded:', df.shape)
```

Cell 2 — Basic Tabular Features

Extract simple but powerful features from each transaction's attributes.

```
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek
df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
df['is_night'] = ((df['hour'] >= 22) | (df['hour'] <= 5)).astype(int)
df['log_amount'] = np.log1p(df['amount'])

# Amount buckets (captures threshold-avoidance fraud)
df['near_limit'] = ((df['amount'] >= 95000) &
(df['amount'] < 100000)).astype(int)

# Transaction type encoding
df['is_p2p'] = (df['txn_type'] == 'P2P').astype(int)
df['is_merchant'] = (df['txn_type'] == 'MERCHANT').astype(int)

print('Basic features added:', ['hour', 'day_of_week', 'is_weekend',
'is_night', 'log_amount', 'near_limit'])
```

Cell 3 — Velocity Features (Rolling Windows)

Velocity features count how many transactions an account made in the last 1 hour, 6 hours, and 24 hours. These are the most important fraud signals — a SIM-swap attacker will generate 30+ transactions in under an hour.

```
df = df.sort_values('timestamp')

def compute_velocity(df, id_col, windows_hours):
    """Count transactions per account in rolling time windows"""
    df = df.copy()
    df['ts_epoch'] = df['timestamp'].astype(np.int64) // 10**9 # Unix seconds

    for h in windows_hours:
        window_sec = h * 3600
        col = f'veLOCITY_{h}h'
        counts = []
        for i, row in df.iterrows():
            t = row['ts_epoch']
            same_acc = df[df[id_col] == row[id_col]]
            cnt = ((same_acc['ts_epoch'] >= t - window_sec) &
                    (same_acc['ts_epoch'] <= t)).sum()
            counts.append(cnt)
        df[col] = counts
    return df

# Note: for 50K rows this takes ~2 minutes
# For faster execution, use the vectorised version below
df['velocity_1h'] = df.groupby('sender_id')['timestamp'].transform(
    lambda x: x.expanding().count())
df['velocity_24h'] = df['velocity_1h'] # simplified for demo

print('Velocity features added')

df[['sender_id', 'amount', 'velocity_1h']].head(5)
```

Cell 4 — NetworkX Graph Features

This is what makes the project unique. You model the entire transaction history as a directed graph where accounts are nodes and transactions are edges. Mule accounts have very high in-degree (many people sending to them) and are detectable via PageRank — they're structurally important in the fraud network.

```
# Build directed transaction graph
G = nx.DiGraph()

# Add all accounts as nodes
for acc_id in df['sender_id'].unique():
    G.add_node(acc_id)
```

```

# Add transactions as edges (weighted by amount)
for _, row in df.iterrows():
    if G.has_edge(row['sender_id'], row['receiver_id']):
        G[row['sender_id']][row['receiver_id']]['weight'] += row['amount']
    else:
        G.add_edge(row['sender_id'], row['receiver_id'],
                    weight=row['amount'])

print(f'Graph: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges')

# Compute graph features per node
pagerank = nx.pagerank(G, weight='weight')
in_degree = dict(G.in_degree())
out_degree = dict(G.out_degree())

# Add to dataframe
df['sender_pagerank'] = df['sender_id'].map(pagerank).fillna(0)
df['sender_in_degree'] = df['sender_id'].map(in_degree).fillna(0)
df['sender_out_degree'] = df['sender_id'].map(out_degree).fillna(0)
df['recv_in_degree'] = df['receiver_id'].map(in_degree).fillna(0)

print('Graph features added')
print('Top 5 by PageRank (likely mule accounts):')
top_nodes = sorted(pagerank.items(), key=lambda x: x[1], reverse=True)[:5]
for node, score in top_nodes:
    print(f' {node}: PageRank={score:.6f}')

```

Cell 5 — Save Final Feature Matrix

```

feature_cols = [
    'hour', 'day_of_week', 'is_weekend', 'is_night',
    'log_amount', 'near_limit', 'is_p2p', 'is_merchant',
    'velocity_1h', 'velocity_24h',
    'sender_pagerank', 'sender_in_degree', 'sender_out_degree', 'recv_in_degree',
    'is_fraud'
]

df_features = df[feature_cols].dropna()
df_features.to_csv('../data/processed/features.csv', index=False)

print(f'Feature matrix saved: {df_features.shape}')
print(f'Fraud rate: {df_features.is_fraud.mean()*100:.2f}%')
df_features.describe()

```

Chapter 5

Model Training & Explainability

XGBoost classifier, SHAP plots, MLflow experiment tracking

Create notebooks/03_model_training.ipynb. You'll train an XGBoost classifier, evaluate it with the right metrics for imbalanced fraud data, generate SHAP explainability plots, and log everything to MLflow.

Cell 1 — Load Features and Split Data

```
import pandas as pd
import numpy as np
import xgboost as xgb
import shap
import mlflow
import mlflow.xgboost
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report,
                             precision_recall_curve, average_precision_score,
                             confusion_matrix, ConfusionMatrixDisplay)
import joblib, os

warnings.filterwarnings('ignore')

df = pd.read_csv('../data/processed/features.csv')

FEATURES = [c for c in df.columns if c != 'is_fraud']
X = df[FEATURES]
y = df['is_fraud']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print(f'Train: {X_train.shape}, Test: {X_test.shape}')
print(f'Fraud in test: {y_test.sum()} ({y_test.mean()*100:.1f}%)')
```

Cell 2 — Train XGBoost with MLflow Tracking

scale_pos_weight handles the class imbalance — it tells XGBoost to weight the minority (fraud) class more heavily during training.

```
# Class imbalance ratio – tells XGBoost to weight fraud cases higher
scale_pos = (y_train == 0).sum() / (y_train == 1).sum()
```

```
print(f'scale_pos_weight: {scale_pos:.1f}x')

mlflow.set_experiment('upi_fraud_detection')

with mlflow.start_run(run_name='xgboost_v1'):

    params = {
        'n_estimators': 300,
        'max_depth': 6,
        'learning_rate': 0.05,
        'subsample': 0.8,
        'colsample_bytree': 0.8,
        'scale_pos_weight': scale_pos,
        'random_state': 42,
        'eval_metric': 'aucpr',
    }

    mlflow.log_params(params)

    model = xgb.XGBClassifier(**params)
    model.fit(X_train, y_train,
              eval_set=[(X_test, y_test)],
              verbose=50)

    # Predict probabilities (NOT hard labels)
    y_proba = model.predict_proba(X_test)[:,-1]

    # Choose operating threshold: Precision >= 0.85
    prec, rec, thresholds = precision_recall_curve(y_test, y_proba)
    valid = [(p,r,t) for p,r,t in zip(prec,rec,thresholds) if p >= 0.85]
    best_t = valid[0][2] if valid else 0.5
    y_pred = (y_proba >= best_t).astype(int)

    pr_auc = average_precision_score(y_test, y_proba)
    mlflow.log_metrics({'pr_auc': pr_auc, 'threshold': best_t})

    print(f'PR-AUC: {pr_auc:.4f}')
    print(f'Operating threshold: {best_t:.3f}')
    print(classification_report(y_test, y_pred,
                                target_names=['Legit', 'Fraud']))

    # Save model
    os.makedirs('../models', exist_ok=True)
    joblib.dump(model, '../models/xgb_fraud_model.pkl')
    joblib.dump(best_t, '../models/threshold.pkl')
```

```
jjoblib.dump(FEATURES, '../models/feature_names.pkl')  
mlflow.xgboost.log_model(model, 'model')  
print('Model saved to models/')
```

Cell 3 — SHAP Explainability Plots

SHAP (SHapley Additive exPlanations) shows which features drove each prediction. This is your most impressive interview output — it shows you understand model interpretability, which is mandatory in regulated fintech environments.

```
explainer = shap.TreeExplainer(model)  
shap_values = explainer.shap_values(X_test[:500]) # sample for speed  
  
# Global feature importance — shows overall model behaviour  
plt.figure(figsize=(10, 6))  
shap.summary_plot(shap_values, X_test[:500], show=False)  
plt.title('SHAP Feature Importance — UPI Fraud Model')  
plt.tight_layout()  
plt.savefig('../models/shap_summary.png', dpi=150)  
plt.show()  
  
# Waterfall plot for one fraud transaction  
fraud_idx = np.where(y_test.values[:500] == 1)[0][0]  
shap.waterfall_plot(  
    shap.Explanation(  
        values=shap_values[fraud_idx],  
        base_values=explainer.expected_value,  
        data=X_test.iloc[fraud_idx],  
        feature_names=FEATURES  
    )  
)
```

What SHAP tells you

The summary plot shows which features matter most globally (e.g., sender_pagerank and velocity_1h will likely appear at the top). The waterfall plot shows for ONE specific transaction why it was flagged — e.g., 'flagged because velocity_1h=47 (very high) and sender_pagerank=0.0023 (top 5% of graph)'. This is what you show in your interview — it tells a story.

View MLflow Dashboard

To see your experiment runs tracked in MLflow, open a new terminal and run:

```
# In VS Code terminal (make sure venv is active):  
mlflow ui
```

```
# Then open your browser and go to:  
# http://localhost:5000  
  
# You'll see your experiment runs with PR-AUC scores, parameters,  
# and saved model artifacts — looks very professional on screen recording
```


FastAPI Scoring Engine

FastAPI is a modern Python web framework that automatically generates API documentation. You'll create a `/predict` endpoint that accepts a transaction, runs it through the model, and returns a fraud score + top SHAP factors. Create the file `api/main.py`.

[illegible]

[illegible]

Run the API

Open VS Code terminal and run:

```
cd api
uvicorn main:app --reload --port 8000
```

```
# --reload means the server restarts when you save main.py
# You'll see: 'Uvicorn running on http://127.0.0.1:8000'

# Open browser → http://localhost:8000/docs
# You'll see the auto-generated Swagger UI – shows all your endpoints
# You can test /predict directly from the browser interface
```

Swagger UI is a major portfolio plus

FastAPI auto-generates interactive API docs at /docs. In your interview demo or README screenshots, showing the Swagger UI makes the project look production-ready. It's also how you test your API without writing extra code.

Streamlit Dashboard & Live Stream

The Streamlit dashboard is your visual demo layer — it's what you show in interviews and put in your README GIF. Create dashboard/app.py.

[illegible]

```
st.divider()

# ■■ Charts row ████████████████████████████████████████████████████████████████████████████

c1, c2 = st.columns(2)

if len(df) > 0:

    df['timestamp'] = pd.to_datetime(df['timestamp'])

    df['hour'] = df['timestamp'].dt.hour

    with c1:

        st.subheader('Fraud Score Distribution')

        fig = px.histogram(df, x='fraud_score', color='is_fraud',
                            nbins=40, color_discrete_map={0:'#2C7BE5',1:'#E63946'})

        st.plotly_chart(fig, use_container_width=True)

    with c2:

        st.subheader('Fraud Rate by Hour')

        hourly = df.groupby('hour')['is_fraud'].mean().reset_index()

        fig2 = px.bar(hourly, x='hour', y='is_fraud',
                       color='is_fraud', color_continuous_scale='RdYlGn_r')

        st.plotly_chart(fig2, use_container_width=True)

# ■■ Live fraud queue ████████████████████████████████████████████████████████████████████████████

st.subheader('Live Fraud Queue (Latest 20 Flagged)')

if len(df) > 0:

    flagged = df[df['is_fraud']==1].tail(20)[

        ['txn_id','amount','sender_vpa','fraud_score','top_reason','timestamp']]

    st.dataframe(flagged, use_container_width=True)

else:

    st.info('No data yet – start simulate_stream.py in the terminal')

# ■■ Manual scoring form ████████████████████████████████████████████████████████████████████████████

st.divider()

st.subheader('Manual Transaction Scoring')

with st.form('score_form'):

    col_a, col_b, col_c = st.columns(3)

    txn_id = col_a.text_input('Transaction ID', 'TXN_TEST_001')

    amount = col_b.number_input('Amount (Rs.)', 100.0, 100000.0, 5000.0)

    vpa = col_c.text_input('Sender VPA', 'test@okicici')

    col_d, col_e, col_f = st.columns(3)

    velocity = col_d.slider('Velocity 1h (txns in last hour)', 1, 60, 2)

    pagerank = col_e.number_input('Sender PageRank', 0.0, 1.0, 0.0001)
```

```

hour = col_f.slider('Hour of Day', 0, 23, 14)
submitted = st.form_submit_button('Score Transaction')

if submitted:
    payload = {
        'txn_id': txn_id, 'sender_vpa': vpa, 'amount': amount,
        'hour': hour, 'day_of_week': 1, 'is_weekend': 0, 'is_night': 0,
        'near_limit': 1 if amount >= 95000 else 0,
        'is_p2p': 1, 'is_merchant': 0,
        'velocity_1h': velocity, 'velocity_24h': velocity * 3,
        'sender_pagerank': pagerank, 'sender_in_degree': 5,
        'sender_out_degree': velocity, 'recv_in_degree': 10,
    }
    resp = requests.post('http://localhost:8000/predict', json=payload)
    result = resp.json()
    risk_color = {'HIGH': '■', 'MEDIUM': '■', 'LOW': '■'}
    st.metric('Fraud Score', result['fraud_score'])
    st.write(f"{risk_color[result['risk_label']] Risk: {result['risk_label']}")
    st.write('Top reasons:', result['top_reasons'])

    # Auto-refresh every 5 seconds when stream is running
    st.caption('Dashboard auto-refreshes every 5 seconds')
    time.sleep(5)
    st.rerun()

```

simulate_stream.py — Live Transaction Simulator

This script fires one synthetic transaction per second at the FastAPI endpoint, making the dashboard update live. Create it in the project root.

```

import requests, random, time, uuid
import numpy as np
from datetime import datetime

API_URL = 'http://localhost:8000/predict'

def make_transaction(is_fraud_scenario=False):
    hour = datetime.now().hour
    amount = np.random.lognormal(6.5, 1.2)
    amount = min(round(amount, 2), 100000)

    if is_fraud_scenario:
        # Simulate a mule account being hit hard

```

```
return {
    'txn_id': str(uuid.uuid4())[:12],
    'sender_vpa': f'fraud.acc{random.randint(1,50)}@paytm',
    'amount': round(random.uniform(80000, 99999), 2),
    'hour': hour,
    'day_of_week': datetime.now().weekday(),
    'is_weekend': 1 if datetime.now().weekday()>=5 else 0,
    'is_night': 1 if hour>=22 or hour<=5 else 0,
    'near_limit': 1,
    'is_p2p': 1,
    'is_merchant': 0,
    'velocity_1h': random.randint(25, 55),
    'velocity_24h': random.randint(60, 120),
    'sender_pagerank': random.uniform(0.005, 0.02),
    'sender_in_degree': random.randint(20, 80),
    'sender_out_degree': random.randint(30, 90),
    'recv_in_degree': random.randint(50, 200),
}
else:
    return {
        'txn_id': str(uuid.uuid4())[:12],
        'sender_vpa': f'normal.user{random.randint(1,4900)}@okicici',
        'amount': amount,
        'hour': hour,
        'day_of_week': datetime.now().weekday(),
        'is_weekend': 1 if datetime.now().weekday()>=5 else 0,
        'is_night': 1 if hour>=22 or hour<=5 else 0,
        'near_limit': 0,
        'is_p2p': random.choice([0,1]),
        'is_merchant': random.choice([0,1]),
        'velocity_1h': random.randint(1, 5),
        'velocity_24h': random.randint(2, 15),
        'sender_pagerank': random.uniform(0.00001, 0.0005),
        'sender_in_degree': random.randint(1, 8),
        'sender_out_degree': random.randint(1, 10),
        'recv_in_degree': random.randint(1, 20),
    }
```



```
print('Starting UPI transaction stream... (Ctrl+C to stop)')
count = 0

while True:
    # 10% of transactions are fraud scenarios
    is_fraud = random.random() < 0.10
    txn = make_transaction(is_fraud_scenario=is_fraud)

    try:
        r = requests.post(API_URL, json=txn, timeout=2)
        result = r.json()
        count += 1

        label = '■ FRAUD' if result['is_fraud'] else '■ OK'
        print(f'[{count:04d}] {label} | Score: {result["fraud_score"]:.4f}'
              f' | Amount: Rs.{txn["amount"]:.0f}'
              f' | {txn["sender_vpa"][:25]}')
    except Exception as e:
        print(f'Error: {e} - is the API running?')

    time.sleep(1) # 1 transaction per second
```

Running Everything Together

Open three terminals in VS Code (click the + icon in the terminal panel):

```
# Terminal 1 – FastAPI (run first)
cd api && uvicorn main:app --reload --port 8000

# Terminal 2 – Streamlit dashboard
cd dashboard && streamlit run app.py

# Terminal 3 – Live stream (run last)
python simulate_stream.py

# Now open browser:
# http://localhost:8501 → Streamlit dashboard (watch it update live!)
# http://localhost:8000/docs → FastAPI Swagger UI
```

Your project is now complete

With all three running you have: a live transaction stream → real-time API scoring → live-updating dashboard. This is the demo you record as a GIF for your README and show in interviews. Commit everything to GitHub and deploy Streamlit to streamlit.io (free) for a public live URL.

Quick Reference — Common Commands

Task	Command
Activate virtual env (Windows)	<code>venv\\Scripts\\activate</code>
Activate virtual env (Mac/Linux)	<code>source venv/bin/activate</code>
Install new package	<code>pip install package_name</code>
Run FastAPI server	<code>cd api && uvicorn main:app --reload</code>
Run Streamlit dashboard	<code>cd dashboard && streamlit run app.py</code>
Run live stream simulator	<code>python simulate_stream.py</code>
Open MLflow dashboard	<code>mlflow ui</code> (then go to <code>localhost:5000</code>)
Freeze dependencies	<code>pip freeze > requirements.txt</code>
Open Jupyter notebook in VS Code	Click <code>.ipynb</code> file in sidebar
Run a notebook cell	Shift + Enter
Open VS Code terminal	Ctrl+` (Windows) or Cmd+` (Mac)
Command palette	Ctrl+Shift+P or Cmd+Shift+P

You now have a complete, working UPI fraud detection system. Push to GitHub, record a demo GIF, and add the live Streamlit URL to your LinkedIn. When you're ready to build the extended version with Kafka and GraphSAGE, come back.